HW3 - Econ 144

Mason Lee

2025-10-30

Packages

```
library(readr)
library(lattice)
library(foreign)
library(MASS)
library(car)
```

```
## Loading required package: carData
```

```
require(stats)
require(stats4)
```

```
## Loading required package: stats4
```

```
library(KernSmooth)
```

```
## KernSmooth 2.23 loaded
## Copyright M. P. Wand 1997-2009
```

```
library(fastICA)
library(cluster)
library(leaps)
library(mgcv)
```

```
## Loading required package: nlme
```

```
## This is mgcv 1.9-1. For overview type 'help("mgcv-package")'.
```

```
library(rpart)
library(pan)
library(mgcv)
library(DAAG)
```

```
##
## Attaching package: 'DAAG'
```

```
## The following object is masked from 'package:car':
##
##     vif

## The following object is masked from 'package:MASS':
##
##     hills
```

```
library("TTR")
library(tis)
```

```
##
## Attaching package: 'tis'

## The following object is masked from 'package:TTR':
##
##     lags

## The following object is masked from 'package:mgcv':
##
##     ti
```

```
require("datasets")
require(graphics)
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo

##
## Attaching package: 'forecast'

## The following object is masked from 'package:tis':
##
##     easter

## The following object is masked from 'package:nlme':
##
##     getResponse
```

```
require(astsa)
```

```
## Loading required package: astsa

##
## Attaching package: 'astsa'

## The following object is masked from 'package:forecast':
##
##     gas
```

```
library(RColorBrewer)
library(plotrix)
library(tsibble)
```

```
## Registered S3 method overwritten by 'tsibble':
##   method               from
##   as_tibble.grouped_df dplyr
```

```
##
## Attaching package: 'tsibble'
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, union
```

```
library(ggplot2)
library(tidyr)
library(USgas)
library(fpp3)
```

```
## -- Attaching packages --------------------------------------- fpp3 1.0.2 --
```

```
## v tibble      3.2.1     v tsibbledata 0.4.1
## v dplyr       1.1.4     v feasts      0.4.2
## v lubridate   1.9.4     v fable       0.4.1
```

```
## -- Conflicts ------------------------------------------------ fpp3_conflicts --
## x feasts::ACF()         masks nlme::ACF()
## x dplyr::between()      masks tis::between()
## x dplyr::collapse()     masks nlme::collapse()
## x lubridate::date()     masks base::date()
## x lubridate::day()      masks tis::day()
## x dplyr::filter()       masks stats::filter()
## x lubridate::hms()      masks tis::hms()
## x tsibble::intersect()  masks base::intersect()
## x lubridate::interval() masks tsibble::interval()
## x dplyr::lag()          masks stats::lag()
## x lubridate::month()    masks tis::month()
## x lubridate::period()   masks tis::period()
## x lubridate::POSIXct()  masks tis::POSIXct()
## x lubridate::quarter()  masks tis::quarter()
## x dplyr::recode()       masks car::recode()
## x dplyr::select()       masks MASS::select()
## x tsibble::setdiff()    masks base::setdiff()
## x lubridate::today()    masks tis::today()
## x tsibble::union()      masks base::union()
## x lubridate::year()     masks tis::year()
## x lubridate::ymd()      masks tis::ymd()
```

Textbook A

Chapter 7

7.2

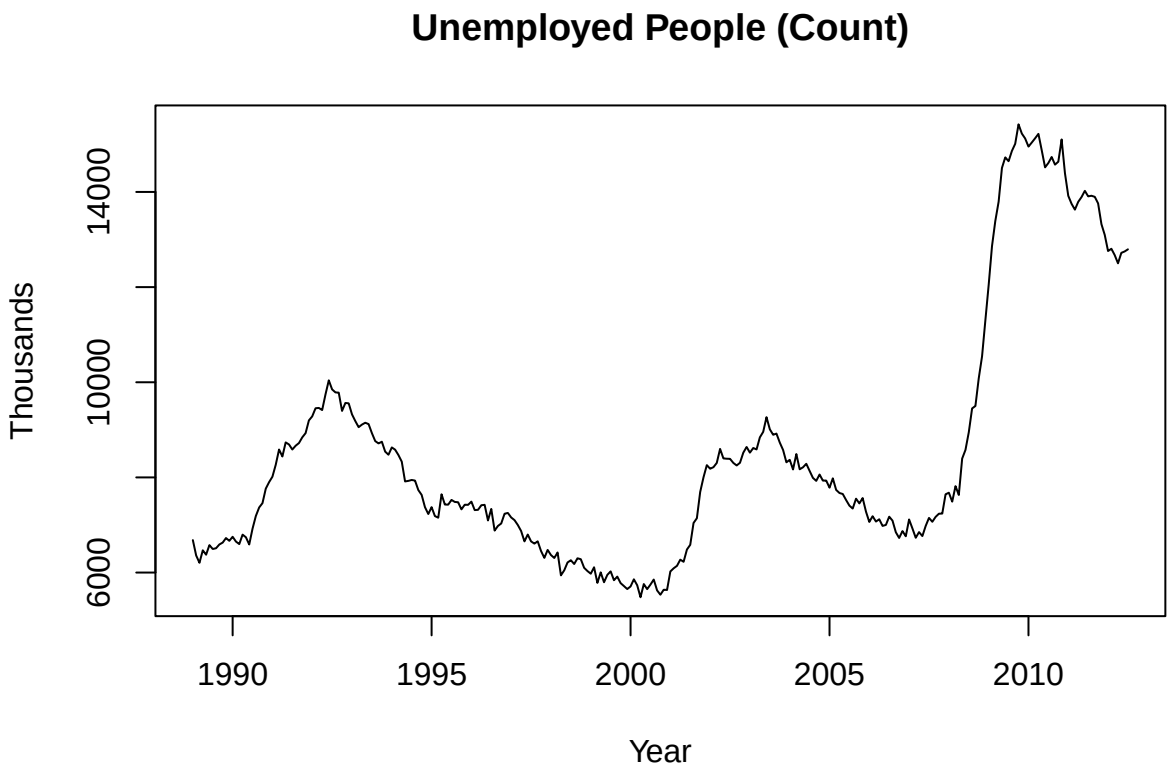
```
data7.2 <- read_csv("72data.csv")
```

```
## Rows: 283 Columns: 2
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): date
## dbl (1): unemployed
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

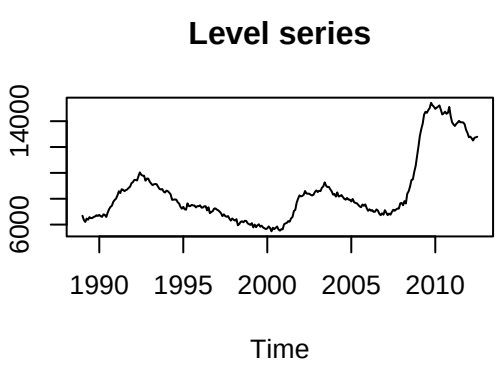
```
head(data7.2)
```

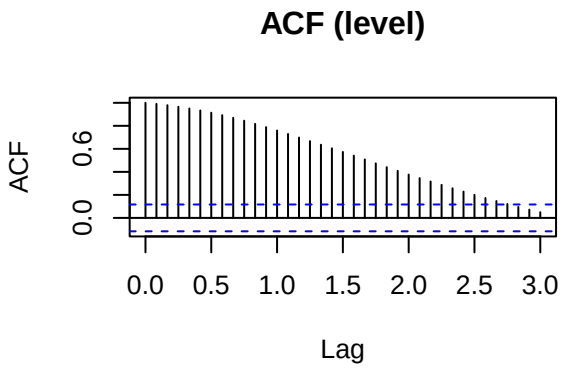
```
## # A tibble: 6 x 2
##   date   unemployed
##   <chr>       <dbl>
## 1 1/1/89       6682
## 2 2/1/89       6359
## 3 3/1/89       6205
## 4 4/1/89       6468
## 5 5/1/89       6375
## 6 6/1/89       6577
```

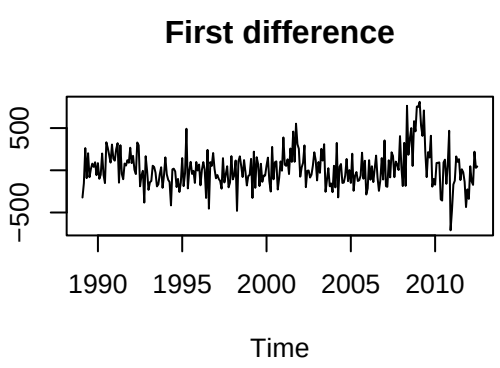
```
data7.2$date <- as.Date(data7.2$date, format = "%m/%d/%y")
data7.2 <- data7.2[order(data7.2$date), ]

start_year  <- as.numeric(format(data7.2$date[1], "%Y"))
start_month <- as.numeric(format(data7.2$date[1], "%m"))
data7.2 <- ts(data7.2$unemployed, start = c(start_year, start_month), frequency = 12)

plot.ts(data7.2,
        main = "Unemployed People (Count)",
        ylab = "Thousands", xlab = "Year")
```

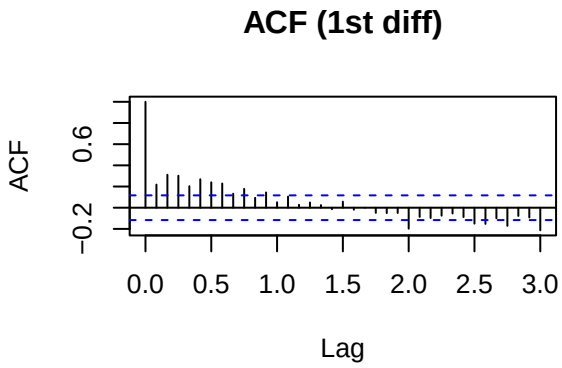



```
data7.2_dif <- diff(data7.2)

par(mfrow = c(2,2))
plot(data7.2,   main = "Level series", ylab = "")
acf(data7.2,    lag.max = 36, main = "ACF (level)")
plot(data7.2_dif, main = "First difference", ylab = "")
acf(data7.2_dif,  lag.max = 36, main = "ACF (1st diff)")
```

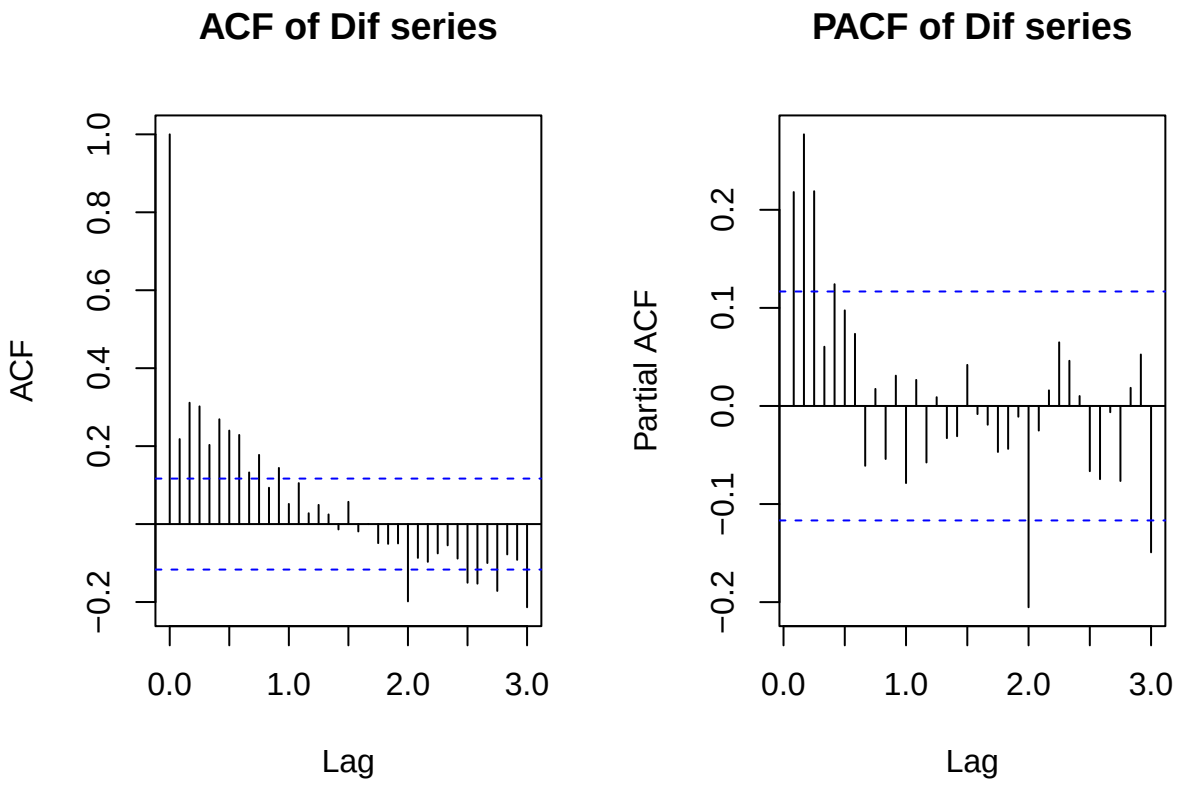









```r
par(mfrow = c(1,1))

par(mfrow = c(1,2))
acf(data7.2_dif,  lag.max = 36, main = "ACF of Dif series")
pacf(data7.2_dif, lag.max = 36, main = "PACF of Dif series")
```




```
par(mfrow = c(1,1))

fit_ar1 <- arima(data7.2, order = c(1,1,0))
fit_ar2 <- arima(data7.2, order = c(2,1,0))
fit_ma1 <- arima(data7.2, order = c(0,1,1))

AIC(fit_ar1, fit_ar2, fit_ma1)
```

```
##         df      AIC
## fit_ar1  2 3834.720
## fit_ar2  3 3813.227
## fit_ma1  2 3839.901
```

```
summary(fit_ar2)
```

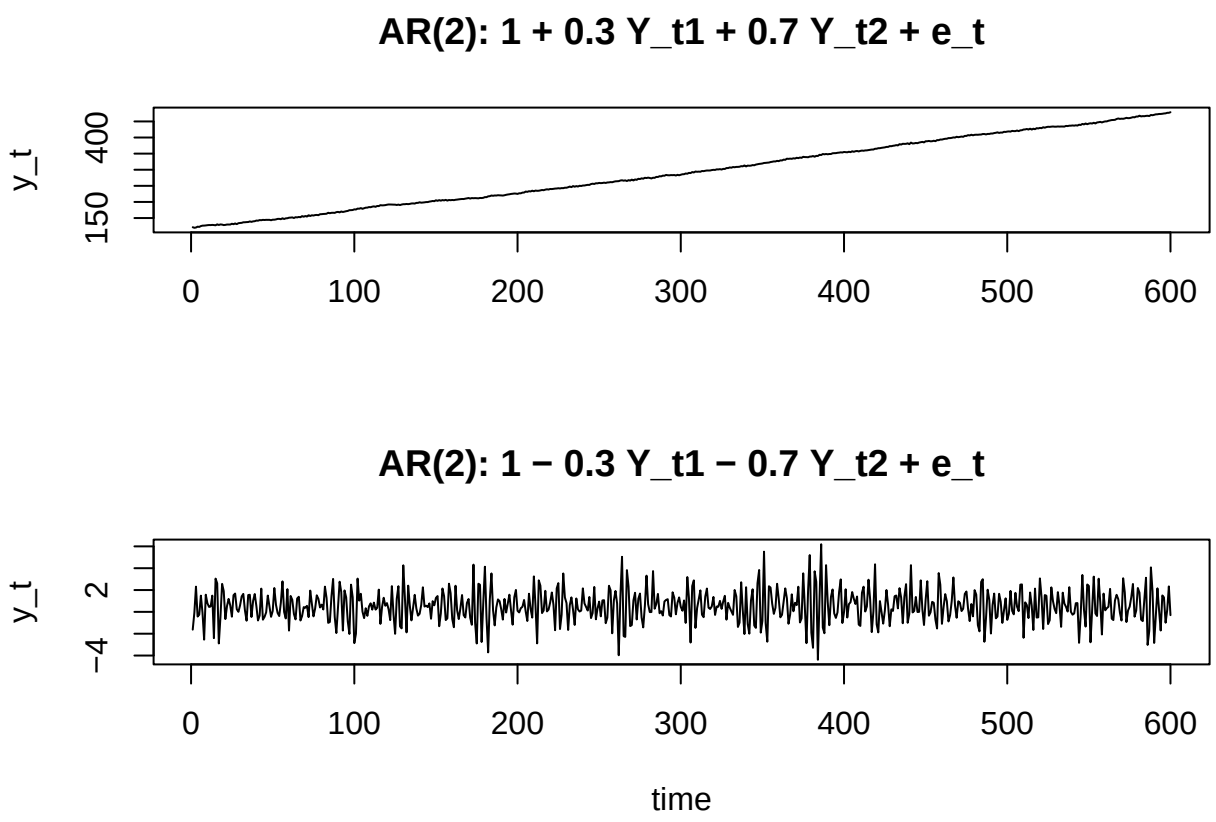
```
##
## Call:
## arima(x = data7.2, order = c(2, 1, 0))
##
## Coefficients:
##          ar1     ar2
##       0.1634  0.2828
## s.e.  0.0571  0.0571
##
## sigma^2 estimated as 42708:  log likelihood = -1903.61,  aic = 3813.23
```

```
##
## Training set error measures:
##                     ME     RMSE      MAE       MPE     MAPE      MASE
## Training set 12.24398 206.2946 162.0193 0.1188639 1.972698 0.9702168
##                      ACF1
## Training set -0.06775919
```

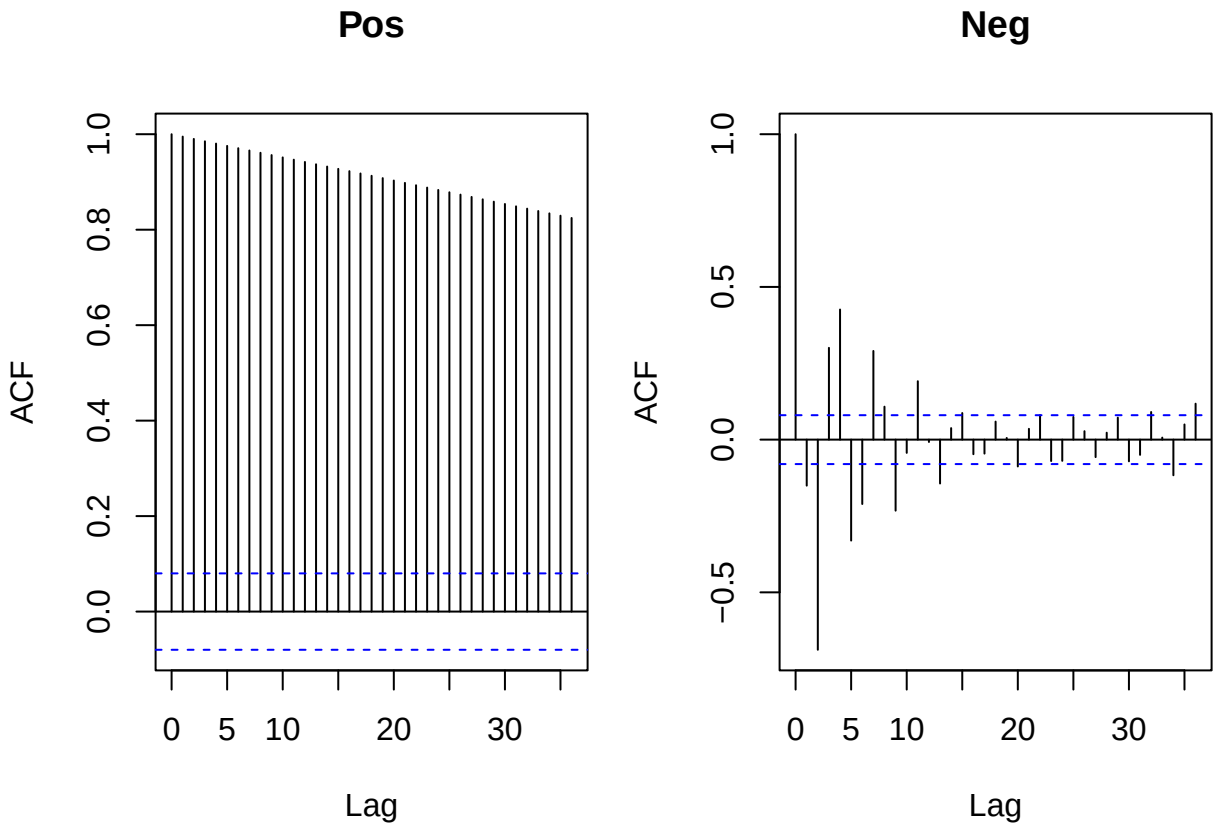
Based on the above results, I do think an autoregressive process can effectively explain the dependence of the series. The slow decaying nature of the ACF plots signal a non-stationary plot, and the PACF plot cutting off early is an indication of an AR model (not MA). Based on the AIC values, an arima(2, 1, 0) model is most appropriate for modeling the unemployment time series.

7.5

```r
set.seed(817)

sim_ar2 <- function(phi1, phi2, c = 1, n = 600, burn = 200) {
  e <- rnorm(n + burn, 0, 1)
  y <- numeric(n + burn)
  for (t in 3:(n + burn)) {
    y[t] <- c + phi1 * y[t-1] + phi2 * y[t-2] + e[t]
  }
  ts(y[(burn+1):(n+burn)])
}

y_pos <- sim_ar2( 0.3,  0.7)
y_neg <- sim_ar2(-0.3, -0.7)

par(mfrow = c(2,1), mar = c(4,4,4,1))
plot.ts(y_pos, main = "AR(2): 1 + 0.3 Y_t1 + 0.7 Y_t2 + e_t", ylab = "y_t", xlab = "")
plot.ts(y_neg, main = "AR(2): 1 - 0.3 Y_t1 - 0.7 Y_t2 + e_t", ylab = "y_t", xlab = "time")
```




```
par(mfrow = c(1,2), mar = c(4,4,4,1))
acf(y_pos, lag.max = 36, main = "Pos")
acf(y_neg, lag.max = 36, main = "Neg")
```




```
par(mfrow = c(1,1))
```

Looking at the graphs of the above simulations, it appears that the one with positive coefficients (called Pos), is much more smooth than the one with negative coefficients (Neg). In the AR(2) models, Pos has a very steady, upwards trajectory with slight bumps but no clear seasonal patterns. Neg, on the other hand, appears to have much more of a seasonal trend. It is far from smooth, but the spiky pattern is apparent. It isn't random enough to be a random walk, and there is no strong upwards trend. ACF plots reveal a similar pattern. Pos has a smooth decline, and Neg is much more chaotic. This tells us that Pos is non-stationary and Neg is stationary.

7.6

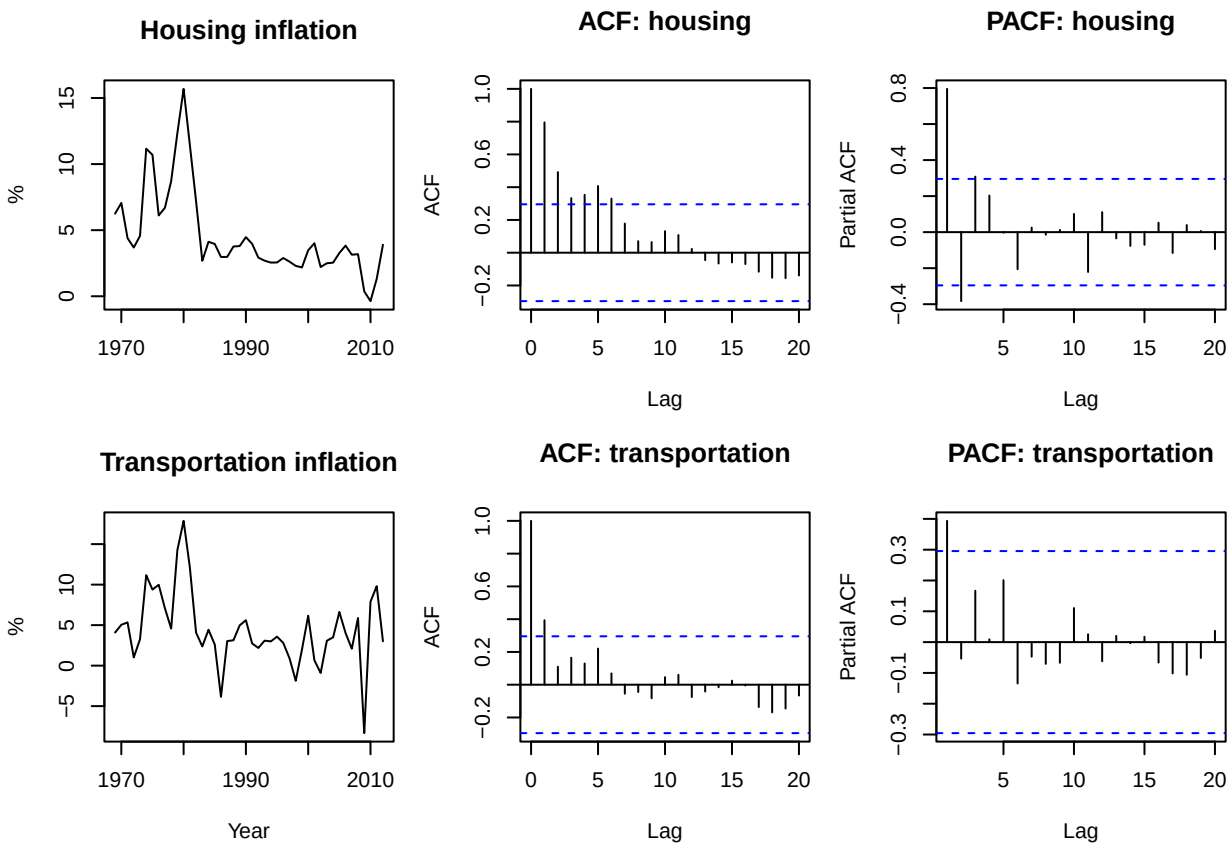
```
data7.6 <- read_csv("76data.csv")
```

```
## Rows: 45 Columns: 5
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): date
## dbl (4): housingCPI, housingInflation, transportationCPI, transportationInfl...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
head(data7.6)
```

```
## # A tibble: 6 x 5
##   date   housingCPI housingInflation transportationCPI transportationInflation
##   <chr>       <dbl>            <dbl>             <dbl>                   <dbl>
## 1 1/1/67       30.8            NA                 33.3                   NA
## 2 1/1/68       32               3.90              34.3                    3.00
## 3 1/1/69       34               6.25              35.7                    4.08
## 4 1/1/70       36.4             7.06              37.5                    5.04
## 5 1/1/71       38               4.40              39.5                    5.33
## 6 1/1/72       39.4             3.68              39.9                    1.01
```

```r
data7.6$date <- as.Date(data7.6$date, format = "%m/%d/%y")
data7.6 <- data7.6[order(data7.6$date), ]

h_infl <- na.omit(data7.6$housingInflation)
t_infl <- na.omit(data7.6$transportationInflation)

start_year_h <- as.numeric(format(data7.6$date[which(!is.na(data7.6$housingInflation))[1]], "%Y"))
start_year_t <- as.numeric(format(data7.6$date[which(!is.na(data7.6$transportationInflation))[1]], "%Y"))

h_ts <- ts(h_infl, start = start_year_h, frequency = 1)
t_ts <- ts(t_infl, start = start_year_t, frequency = 1)

par(mfrow = c(2,3), mar = c(4,4,4,1))
plot.ts(h_ts, main = "Housing inflation", ylab = "%", xlab = "")
acf(h_ts, lag.max = 20, main = "ACF: housing")
pacf(h_ts, lag.max = 20, main = "PACF: housing")

plot.ts(t_ts, main = "Transportation inflation", ylab = "%", xlab = "Year")
acf(t_ts, lag.max = 20, main = "ACF: transportation")
pacf(t_ts, lag.max = 20, main = "PACF: transportation")
```




```
par(mfrow = c(1,1))
```

Housing: AR(2) seems appropriate in this case. The ACF plot has a very steady decay, and when that is paired with a PACF that has a strong spike at 1 (and in this case, negative spike at 2), an AR model is usually a good forecaster.

Transportation: Here, the ACF and PACF both decline quickly. Each graph has a strong first spike, but none of the other lines (with the second ACF line as a small exception) cross the blue significance threshold. This indicates AR may not be the best model, and that it may be better to go with an MA or ARIMA.

7.9

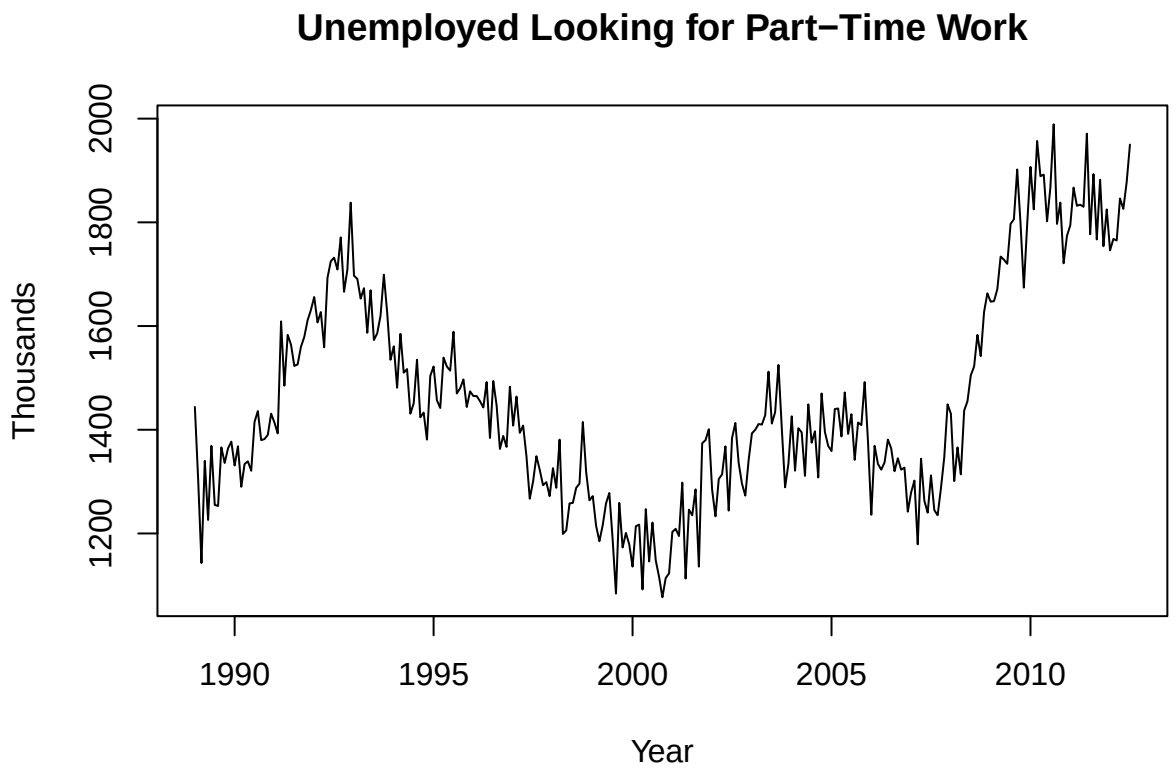
```
data7.9 <- read_csv("79data.csv")
```

```
## Rows: 283 Columns: 2
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): date
## dbl (1): unempl_part
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

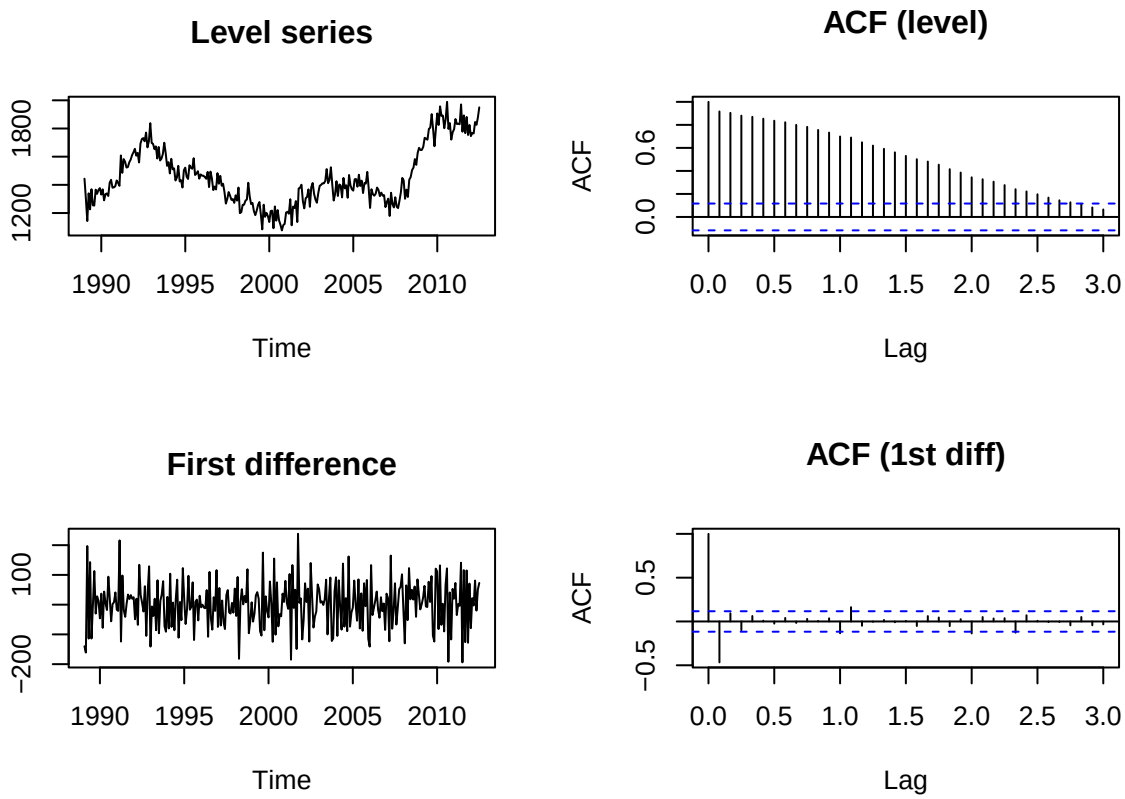
```r
head(data7.9)
```

```
## # A tibble: 6 x 2
##   date   unempl_part
##   <chr>        <dbl>
## 1 1/1/89        1444
## 2 2/1/89        1304
## 3 3/1/89        1143
## 4 4/1/89        1340
## 5 5/1/89        1226
## 6 6/1/89        1369
```

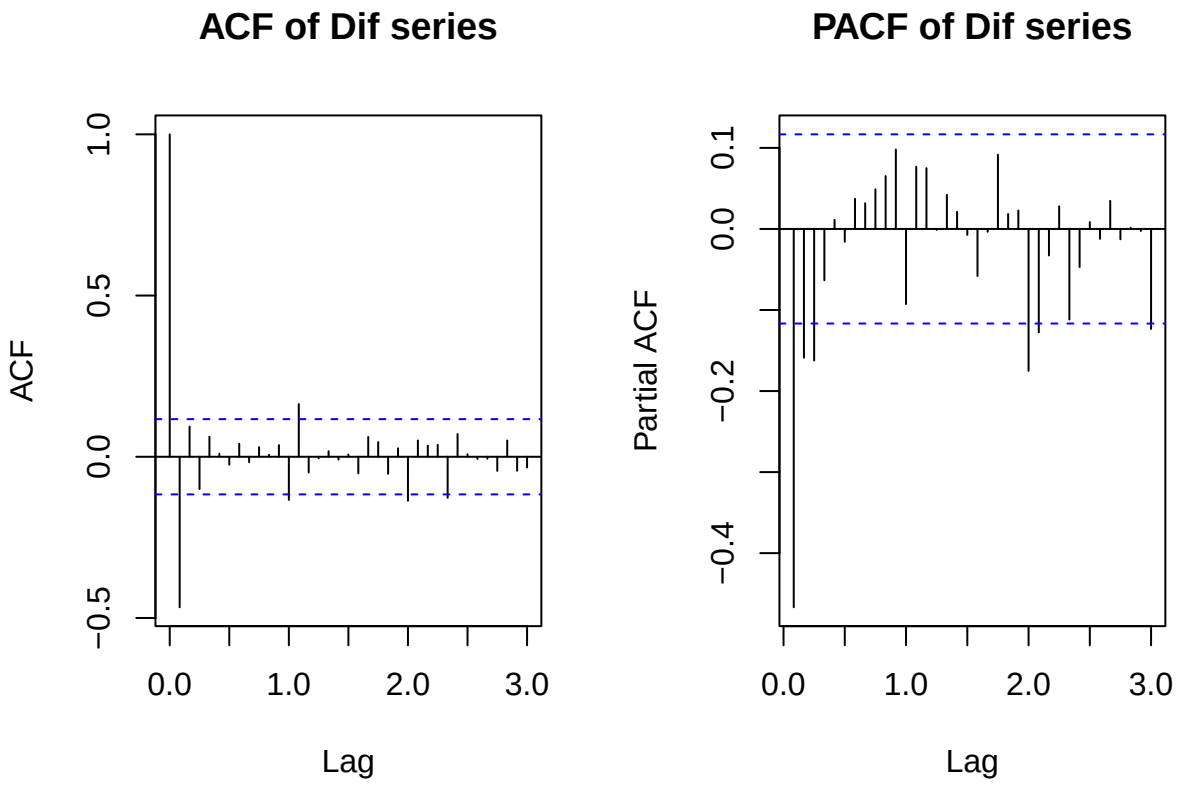
```r
data7.9$date <- as.Date(data7.9$date, format = "%m/%d/%y")
data7.9 <- data7.9[order(data7.9$date), ]

start_year  <- as.numeric(format(data7.9$date[1], "%Y"))
start_month <- as.numeric(format(data7.9$date[1], "%m"))
data7.9_ts  <- ts(data7.9$unempl_part,
                  start = c(start_year, start_month),
                  frequency = 12)

plot.ts(data7.9_ts,
        main = "Unemployed Looking for Part-Time Work",
        ylab = "Thousands", xlab = "Year")
```



```r
data7.9_dif <- diff(data7.9_ts)

par(mfrow = c(2,2))
plot(data7.9_ts,   main = "Level series", ylab = "")
acf (data7.9_ts,   lag.max = 36, main = "ACF (level)")
plot(data7.9_dif,  main = "First difference", ylab = "")
acf (data7.9_dif,  lag.max = 36, main = "ACF (1st diff)")
```



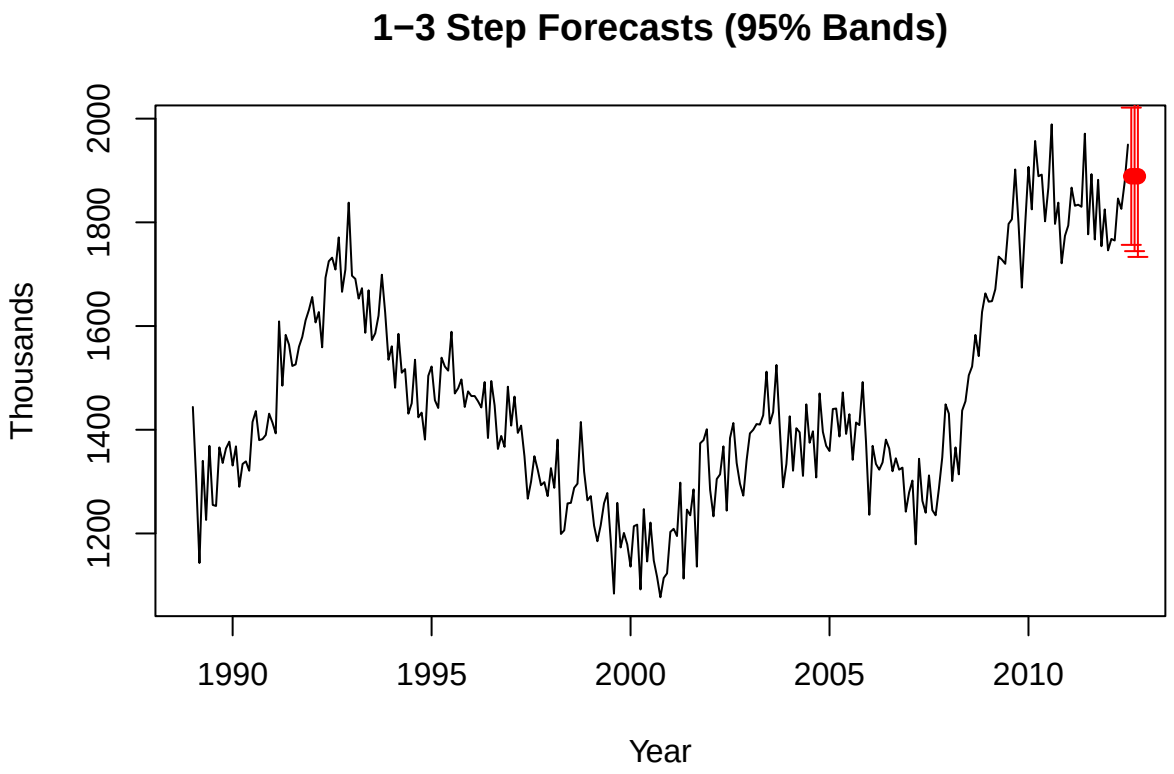

```r
par(mfrow = c(1,1))

par(mfrow = c(1,2))
acf (data7.9_dif,  lag.max = 36, main = "ACF of Dif series")
pacf(data7.9_dif,  lag.max = 36, main = "PACF of Dif series")
```




```
par(mfrow = c(1,1))

fit_ar1 <- arima(data7.9_ts, order = c(1,1,0))  # ARIMA(1,1,0)
fit_ar2 <- arima(data7.9_ts, order = c(2,1,0))  # ARIMA(2,1,0)
fit_ma1 <- arima(data7.9_ts, order = c(0,1,1))  # ARIMA(0,1,1)

AIC(fit_ar1, fit_ar2, fit_ma1)
```

```
##         df      AIC
## fit_ar1  2 3195.133
## fit_ar2  3 3188.338
## fit_ma1  2 3180.473
```

```
best <- predict(fit_ma1, n.ahead = 3)
best_mean <- as.numeric(best$pred)
best_se   <- as.numeric(best$se)
best_lo   <- best_mean - 1.96 * best_se
best_hi   <- best_mean + 1.96 * best_se

plot.ts(data7.9_ts, ylab = "Thousands", xlab = "Year",
        main = "1-3 Step Forecasts (95% Bands)")
last_t <- time(data7.9_ts)[length(data7.9_ts)]
next_idx <- seq(from = last_t + 1/12, by = 1/12, length.out = 3)
points(next_idx, best_mean, pch = 19, col = "red")
arrows(next_idx, best_lo, next_idx, best_hi, angle = 90, code = 3, length = 0.05, col = "red")
```




```
k <- 12
y_full <- data7.9_ts
y_tr   <- window(y_full, end = time(y_full)[length(y_full)-k])
y_te   <- window(y_full, start = time(y_full)[length(y_full)-k+1])

fit_ma1_tr <- arima(y_tr, order = c(0,1,1))
fc_te <- predict(fit_ma1_tr, n.ahead = length(y_te))
errs <- as.numeric(y_te) - as.numeric(fc_te$pred)

data.frame(step = 1:length(errs),
           actual = as.numeric(y_te),
           fcast  = round(as.numeric(fc_te$pred),2),
           error  = round(errs,2))
```

```
##    step actual   fcast  error
## 1     1   1893 1841.57  51.43
## 2     2   1767 1841.57 -74.57
## 3     3   1882 1841.57  40.43
## 4     4   1754 1841.57 -87.57
## 5     5   1825 1841.57 -16.57
## 6     6   1746 1841.57 -95.57
## 7     7   1768 1841.57 -73.57
## 8     8   1765 1841.57 -76.57
## 9     9   1846 1841.57   4.43
## 10   10   1826 1841.57 -15.57
## 11   11   1877 1841.57  35.43
```

```
## 12   12   1950 1841.57 108.43
```

```
c(MAE = mean(abs(errs)), RMSE = sqrt(mean(errs^2)))
```

```
##      MAE     RMSE
## 56.67825 65.48965
```

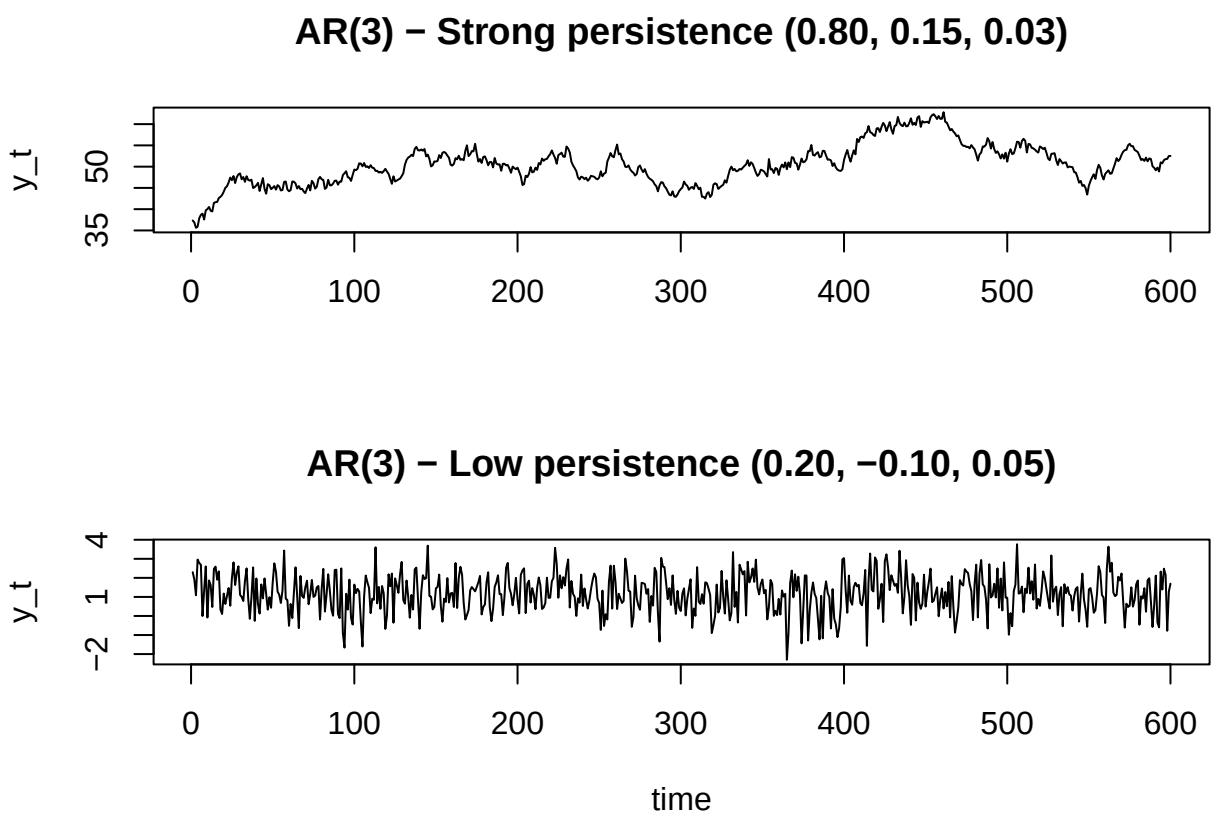
```
mean(errs)
```

```
## [1] -16.65285
```

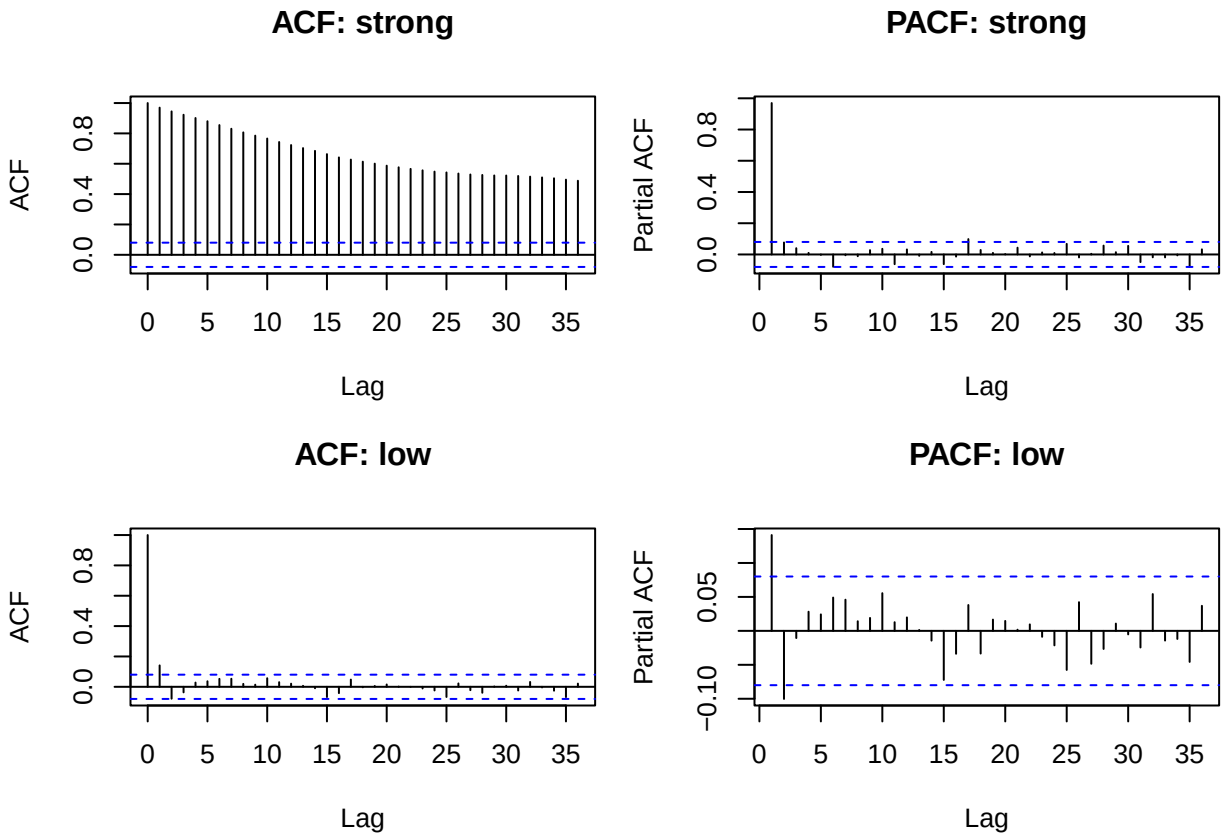
I found MA(1) to be the best model due to its relatively low AIC value. Plotting its 95% CI, we see that using an MA(1) model, we predict that the graph will level out for the next 3 months. The errors tell a slightly different story. Our constant prediction proved very inconsistent when compared to the actual data, and tended to underpredict 70% of the time. Although average doesn't tell you much when it comes to predicting seasonal patterns due to the increasing nature of the data, we were off by -16 on average, indicating a low prediction tendancy.

7.10

```
sim_ar3 <- function(phi1, phi2, phi3, c = 1, n = 600, burn = 200) {
  e <- rnorm(n + burn, 0, 1)
  y <- numeric(n + burn)
  for (t in 4:(n + burn)) {
    y[t] <- c + phi1*y[t-1] + phi2*y[t-2] + phi3*y[t-3] + e[t]
  }
  ts(y[(burn+1):(n+burn)])
}


phi_strong <- c(0.80, 0.15, 0.03)   ## designed to be stationary

phi_low    <- c(0.20, -0.10, 0.05)  ## designed for quick decay

y_strong <- sim_ar3(phi_strong[1], phi_strong[2], phi_strong[3])
y_low    <- sim_ar3(phi_low[1],    phi_low[2],    phi_low[3])

par(mfrow = c(2,1), mar = c(4,4,4,1))
plot.ts(y_strong, main = "AR(3) - Strong persistence (0.80, 0.15, 0.03)", ylab = "y_t", xlab = "")
plot.ts(y_low,    main = "AR(3) - Low persistence (0.20, -0.10, 0.05)",    ylab = "y_t", xlab = "time")
```




```r
par(mfrow = c(2,2), mar = c(4,4,4,1))
acf (y_strong, lag.max = 36, main = "ACF: strong")
pacf(y_strong, lag.max = 36, main = "PACF: strong")
acf (y_low,    lag.max = 36, main = "ACF: low")
pacf(y_low,    lag.max = 36, main = "PACF: low")
```




```
par(mfrow = c(1,1))
```

Above are the two simulated AR(3) models. One was designed to be stationary due to the high sum of coefficients, and one was designed to quickly decay due to the low sum of coefficients. As we can see, the simulation is consisent with the theoretical behavior. The "strong" model has a very slowly decaying ACF, and a PACF that has a large early spike followed by mostly insignificant spikes hovering between the blue bands. The "low" model has an ACF that quickly collapses (much like the "strong" PACF), and a PACF with two large spikes followed by a jumble of insignificant lag values.

Chapter 8

8.1

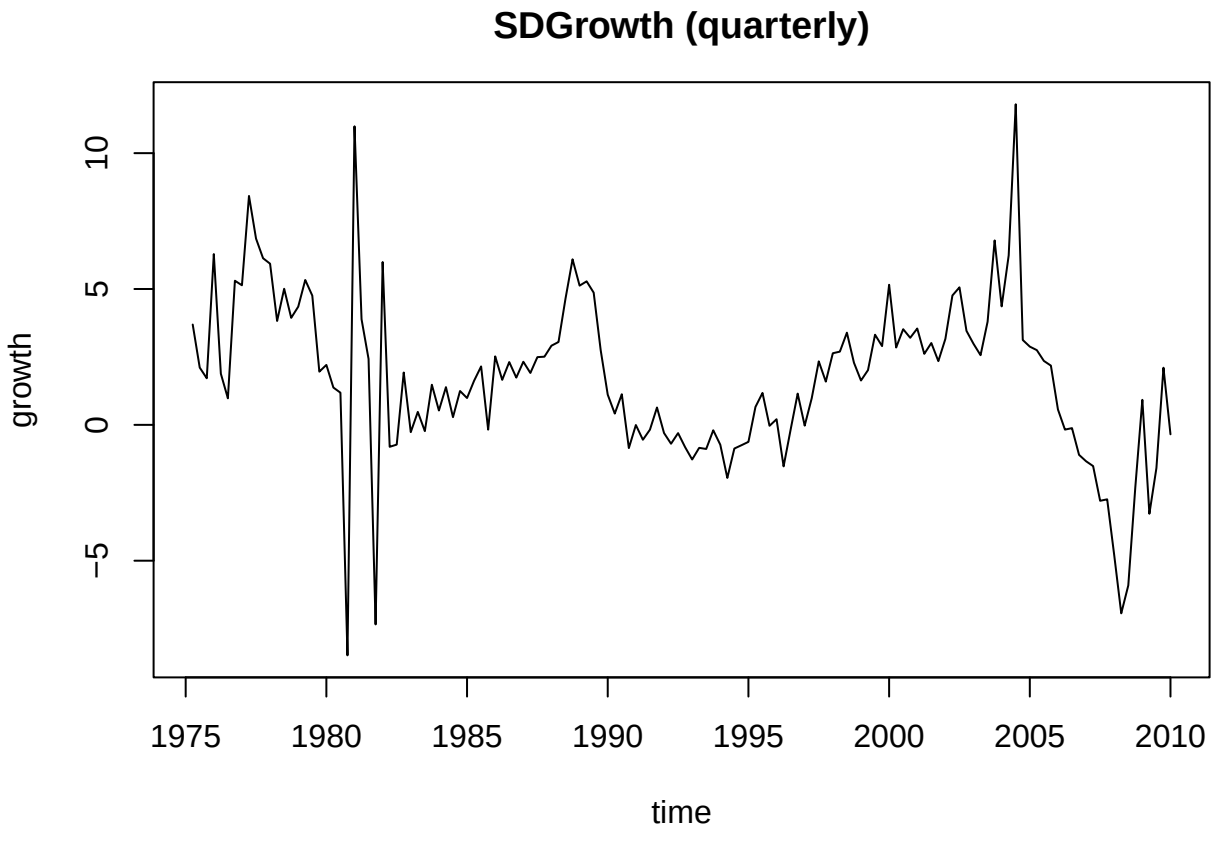
```
data81 <- read_csv("data81.csv")
```

```
## Rows: 184 Columns: 3
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): OBS
## dbl (2): SDindex, SDGrowth
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
head(data81)
```

```
## # A tibble: 6 x 3
##   OBS    SDindex SDGrowth
##   <chr>    <dbl>    <dbl>
## 1 1975Q1    32.5    NA
## 2 1975Q2    33.7     3.69
## 3 1975Q3    34.4     2.11
## 4 1975Q4    35       1.71
## 5 1976Q1    37.2     6.29
## 6 1976Q2    37.9     1.88
```

```r
y_df <- data81[, c("OBS", "SDGrowth")]
y_df <- y_df[!is.na(y_df$SDGrowth), ]


yr0 <- as.integer(substr(y_df$OBS[1], 1, 4))
q0  <- as.integer(substr(y_df$OBS[1], 6, 6))


y <- ts(y_df$SDGrowth, start = c(yr0, q0), frequency = 4)

par(mfrow = c(1,1), mar = c(4,4,3,1))
plot.ts(y, main = "SDGrowth (quarterly)", ylab = "growth", xlab = "time")
```

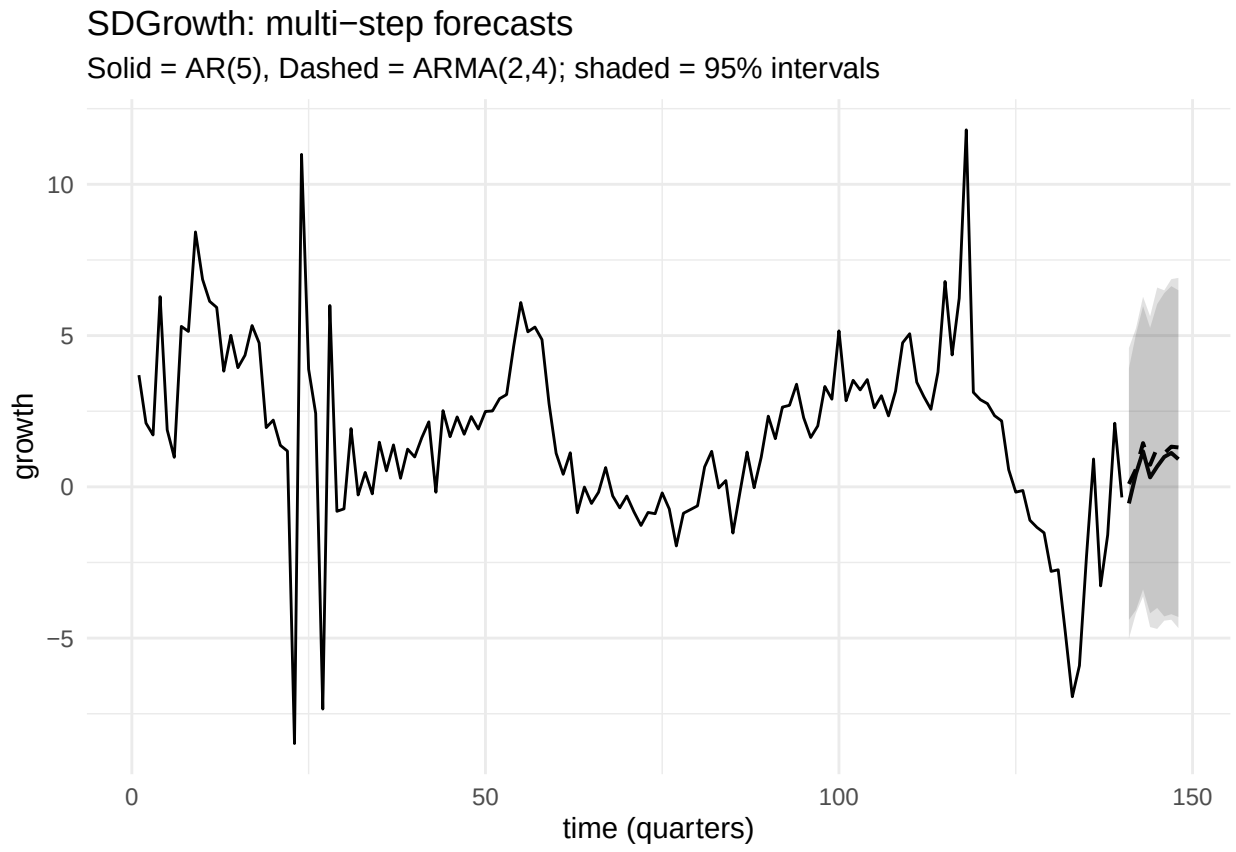


```r
fit_ar5 <- arima(y, order = c(5,0,0), include.mean = TRUE)
fit_arma24 <- arima(y, order = c(2,0,4), include.mean = TRUE)

AIC(fit_ar5, fit_arma24)   # model selection info
```

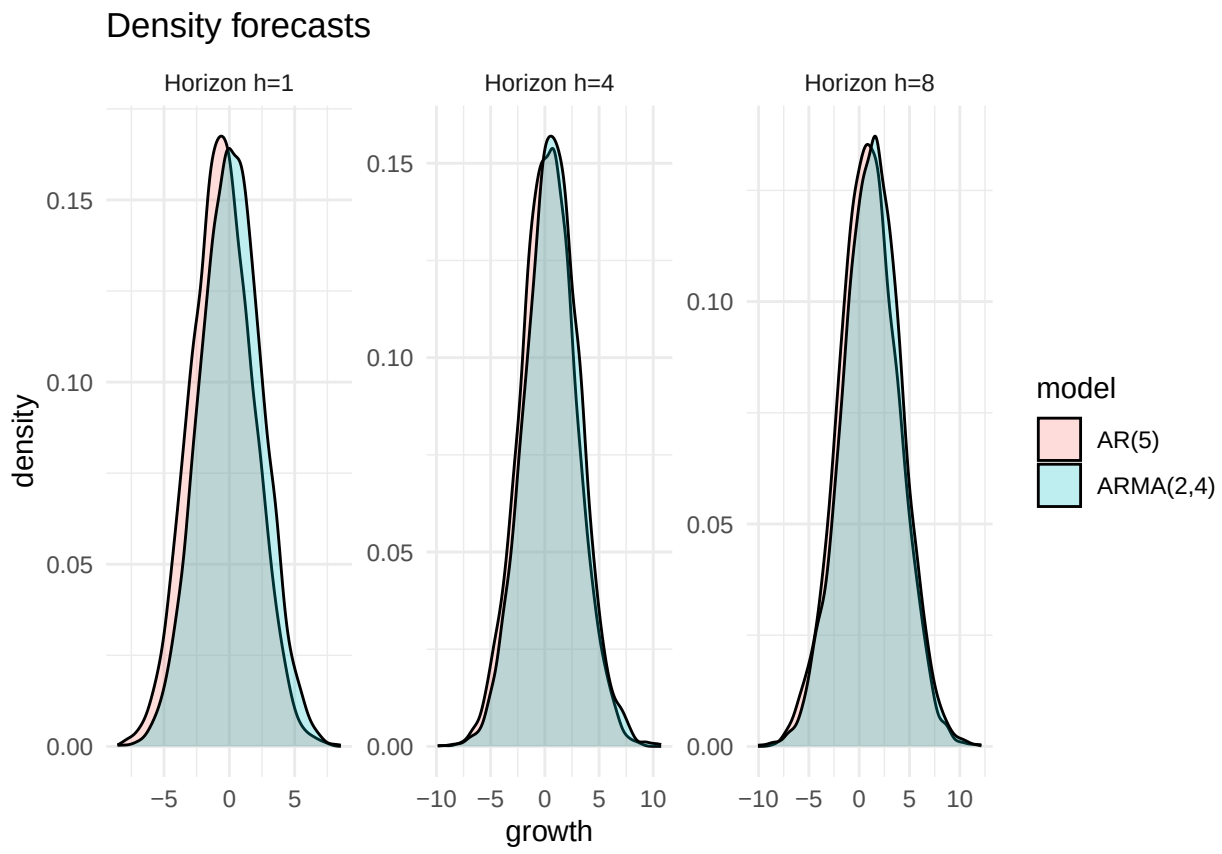
```
##            df      AIC
## fit_ar5     7 643.0346
## fit_arma24  8 646.5985
```

```r
h <- 8

fc_ar5     <- predict(fit_ar5,    n.ahead = h)
fc_arma24  <- predict(fit_arma24, n.ahead = h)

mk_fc_tbl <- function(fc, y) {
  m  <- as.numeric(fc$pred)
  se <- as.numeric(fc$se)
  lo <- m - 1.96*se
  hi <- m + 1.96*se
  # future time index as ts time then map to Date-like labels for ggplot
  t_last <- time(y)[length(y)]
  t_seq  <- seq(from = t_last + 1/frequency(y), by = 1/frequency(y), length.out = length(m))
  # convert to year-quarter labels
  yr <- floor(t_seq)
  q  <- 1 + round((t_seq - yr) * frequency(y))
  data.frame(step = 1:length(m), year = yr, quarter = q, mean = m, lo = lo, hi = hi)
}

fc_tab_ar5    <- mk_fc_tbl(fc_ar5, y)
fc_tab_arma24 <- mk_fc_tbl(fc_arma24, y)


fit_ar5_f     <- forecast::Arima(y, order = c(5,0,0), include.mean = TRUE)
fit_arma24_f  <- forecast::Arima(y, order = c(2,0,4), include.mean = TRUE)

set.seed(81)
Npaths <- 5000   # EDIT IF NEEDED

sim_mat_ar5    <- replicate(Npaths, simulate(fit_ar5_f,    nsim = h, future = TRUE))
sim_mat_arma24 <- replicate(Npaths, simulate(fit_arma24_f, nsim = h, future = TRUE))

pick_h <- intersect(c(1,4,8), 1:h)

dens_df <- do.call(rbind, lapply(pick_h, function(k) {
  rbind(
    data.frame(model = "AR(5)",     h = k, value = sim_mat_ar5[k, ]),
    data.frame(model = "ARMA(2,4)", h = k, value = sim_mat_arma24[k, ])
  )
}))

hist_df <- data.frame(
  t = seq_along(y),
  value = as.numeric(y)
```

```
)

t_hist_last <- nrow(hist_df)
fc_tab_ar5$t  <- t_hist_last + fc_tab_ar5$step
fc_tab_arma24$t <- t_hist_last + fc_tab_arma24$step

ggplot() +
  geom_line(data = hist_df, aes(t, value)) +
  geom_ribbon(data = fc_tab_ar5, aes(t, ymin = lo, ymax = hi), alpha = 0.15) +
  geom_line  (data = fc_tab_ar5, aes(t, mean), linewidth = 0.7) +
  geom_ribbon(data = fc_tab_arma24, aes(t, ymin = lo, ymax = hi), alpha = 0.15) +
  geom_line  (data = fc_tab_arma24, aes(t, mean), linewidth = 0.7, linetype = "dashed") +
  labs(title = "SDGrowth: multi-step forecasts",
       subtitle = "Solid = AR(5), Dashed = ARMA(2,4); shaded = 95% intervals",
       x = "time (quarters)", y = "growth") +
  theme_minimal()
```




```
ggplot(dens_df, aes(x = value, fill = model)) +
  geom_density(alpha = 0.25, adjust = 1.0) +
  facet_wrap(~ h, scales = "free", nrow = 1,
             labeller = labeller(h = function(x) paste0("Horizon h=", x))) +
  labs(title = "Density forecasts", x = "growth", y = "density") +
  theme_minimal()
```




```
comp_tab <- data.frame(
  step = fc_tab_ar5$step,
  ar5_mean = round(fc_tab_ar5$mean, 3),
  ar5_lo95 = round(fc_tab_ar5$lo,   3),
  ar5_hi95 = round(fc_tab_ar5$hi,   3),
  arma24_mean = round(fc_tab_arma24$mean, 3),
  arma24_lo95 = round(fc_tab_arma24$lo,   3),
  arma24_hi95 = round(fc_tab_arma24$hi,   3)
)

comp_tab
```

```
##   step ar5_mean ar5_lo95 ar5_hi95 arma24_mean arma24_lo95 arma24_hi95
## 1    1   -0.548   -5.016    3.920       0.095      -4.398       4.589
## 2    2    0.434   -4.182    5.051       0.587      -4.070       5.243
## 3    3    1.174   -3.625    5.973       1.446      -3.393       6.284
## 4    4    0.312   -4.635    5.259       0.730      -4.179       5.639
## 5    5    0.673   -4.696    6.043       1.292      -4.004       6.589
## 6    6    0.991   -4.424    6.407       1.107      -4.282       6.496
## 7    7    1.121   -4.393    6.635       1.328      -4.209       6.866
## 8    8    0.914   -4.669    6.497       1.302      -4.302       6.907
```

Comparing the two models I forecast (AR(5) and ARIMA(2,4)) to AR(4) from 8.3, we see that there are some key differences. Although all forecast positive growth, AR(2,4) is more optimistic than AR(4), and AR(5) is a bit more pessimistic. The forecast story is generally the same, and the confidence intervals are

almost identical. Regardless of your choice of model, you will get relatively indistinguishable results up to 12 periods out.

8.2

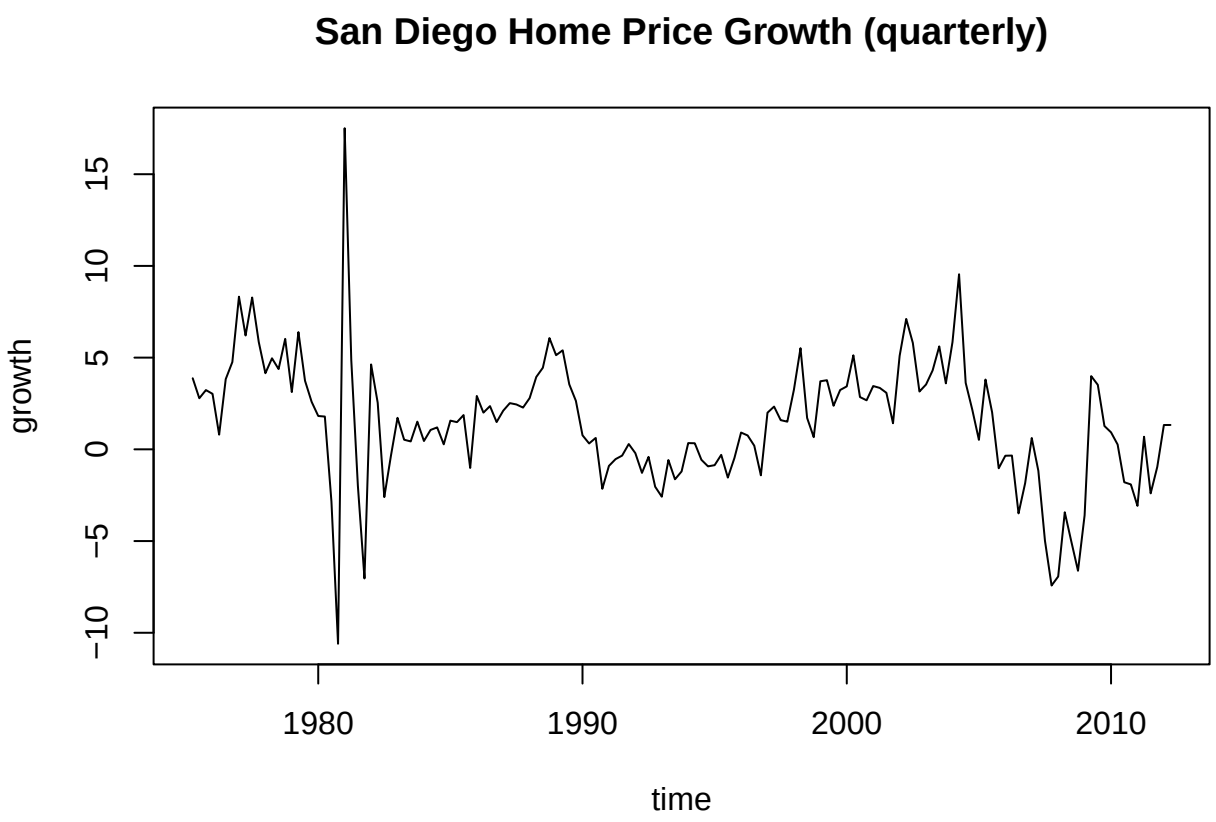
```
data82 <- read_csv("data82.csv")
```

```
## Rows: 184 Columns: 3
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): date
## dbl (2): sd, sdg
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

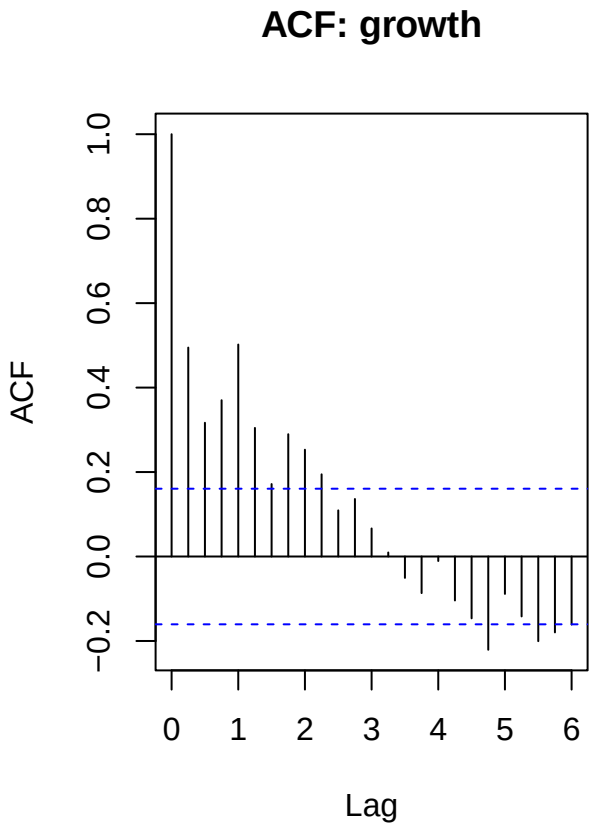
```
head(data82)
```

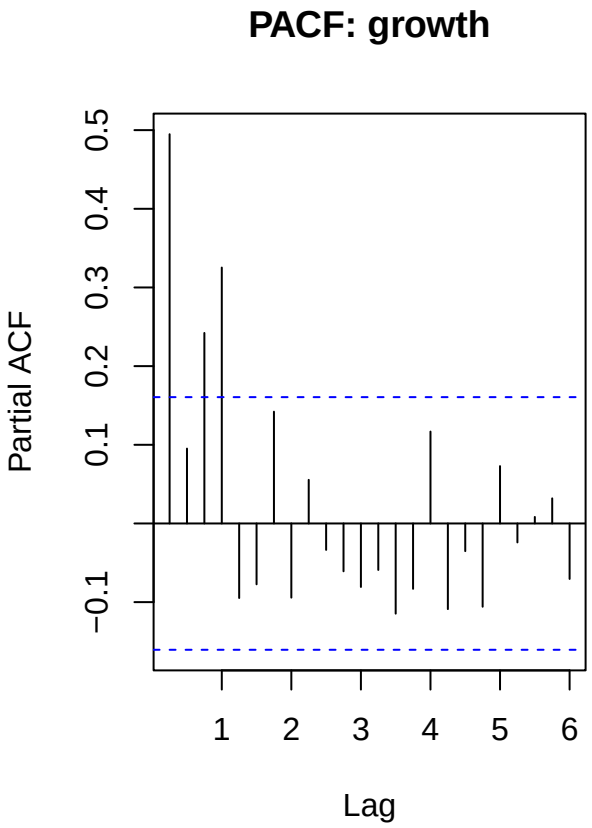
```
## # A tibble: 6 x 3
##   date       sd    sdg
##   <chr>   <dbl>  <dbl>
## 1 1/1/75   16.4 NA
## 2 4/1/75   17.1  3.88
## 3 7/1/75   17.5  2.78
## 4 10/1/75  18.1  3.23
## 5 1/1/76   18.6  3.03
## 6 4/1/76   18.8  0.801
```

```
d <- subset(data82, !is.na(sdg), c(date, sdg))
d$date <- as.Date(d$date, format = "%m/%d/%y")
d <- d[order(d$date), ]

start_year  <- as.numeric(format(d$date[1], "%Y"))
start_quart <- (as.numeric(format(d$date[1], "%m")) - 1) %/% 3 + 1
y <- ts(d$sdg, start = c(start_year, start_quart), frequency = 4)

par(mar = c(4,4,4,1))
plot.ts(y, main = "San Diego Home Price Growth (quarterly)", xlab = "time", ylab = "growth")
```

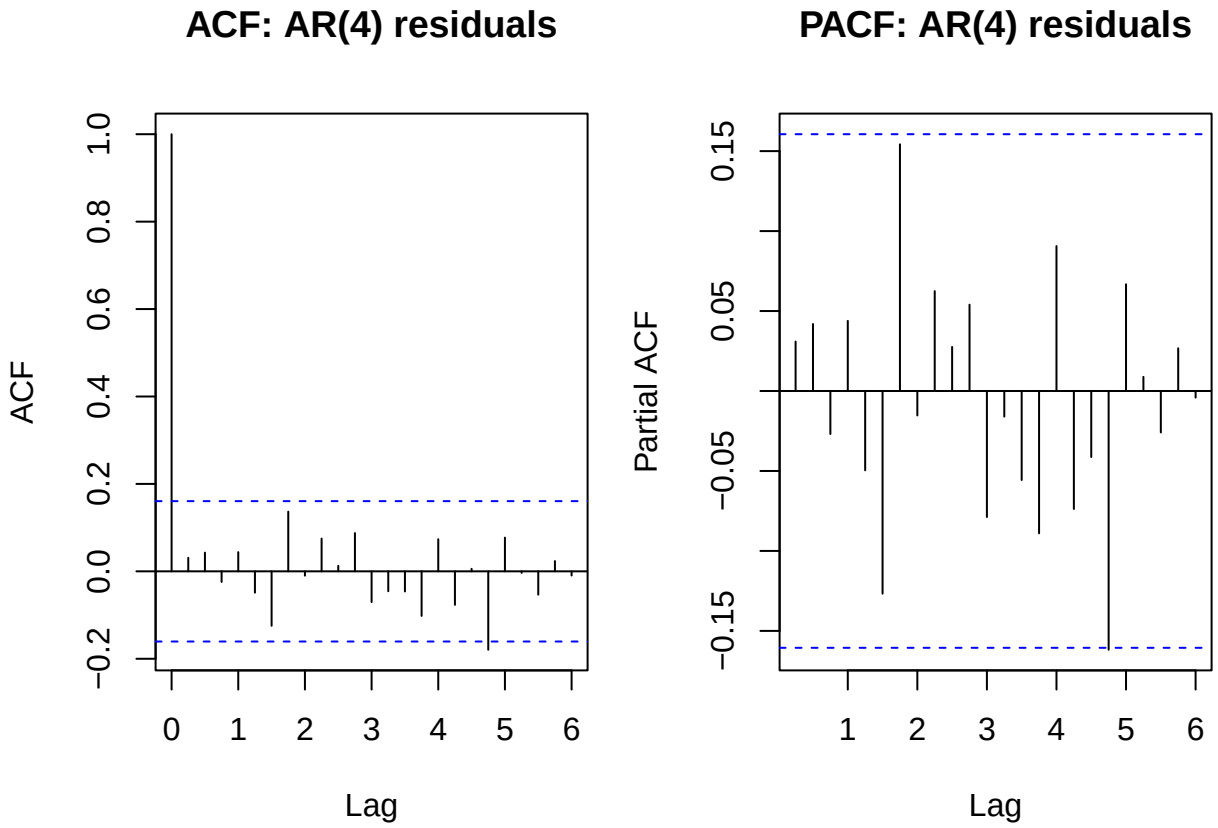



```
par(mfrow = c(1,2), mar = c(4,4,4,1))
acf (y,  lag.max = 24, main = "ACF: growth")
pacf(y,  lag.max = 24, main = "PACF: growth")
```






```r
par(mfrow = c(1,1))

fit_ar4 <- arima(y, order = c(4,0,0), include.mean = TRUE)
resid4  <- residuals(fit_ar4)

par(mfrow = c(1,2), mar = c(4,4,4,1))
acf (resid4, lag.max = 24, main = "ACF: AR(4) residuals")
pacf(resid4, lag.max = 24, main = "PACF: AR(4) residuals")
```




```
par(mfrow = c(1,1))
```

Now that we have the full time series data plotted, we see that AR(4) still fits well. The ACF is slowly decreasing, and the PACF has spikes until lag 4 (year 1), as expected. The residuals also back up this claim, since they seem random and unbiased.

8.5

```
data85 <- read_csv("data85.csv")
```

```
## Rows: 262 Columns: 2
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): DATE
## dbl (1): growth
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
head(data85)
```

```
## # A tibble: 6 x 2
```

```
##   DATE    growth
##   <chr>    <dbl>
## 1 1/1/47    NA
## 2 4/1/47    -0.2
## 3 7/1/47    -0.1
## 4 10/1/47    1.5
## 5 1/1/48     1.6
## 6 4/1/48     1.8
```

```
d <- subset(data85, !is.na(growth), c(DATE, growth))
d$DATE <- as.Date(d$DATE, format = "%m/%d/%y")
d <- d[order(d$DATE), ]
start_year  <- as.numeric(format(d$DATE[1], "%Y"))
start_quart <- (as.numeric(format(d$DATE[1], "%m")) - 1) %/% 3 + 1
y <- ts(d$growth, start = c(start_year, start_quart), frequency = 4)
n <- length(y)

fit_ar1 <- arima(y, order = c(1,0,0), include.mean = TRUE)
fit_ma1 <- arima(y, order = c(0,0,1), include.mean = TRUE)
fit_arma11  <- arima(y, order = c(1,0,1), include.mean = TRUE)
fit_arma21  <- arima(y, order = c(2,0,1), include.mean = TRUE)
fit_arma12  <- arima(y, order = c(1,0,2), include.mean = TRUE)
fit_arma22  <- arima(y, order = c(2,0,2), include.mean = TRUE)
fit_ar4 <- arima(y, order = c(4,0,0), include.mean = TRUE)
fit_ma4 <- arima(y, order = c(0,0,4), include.mean = TRUE)

AIC(fit_ar1, fit_ma1, fit_arma11, fit_arma21, fit_arma12, fit_arma22, fit_ar4, fit_ma4)
```

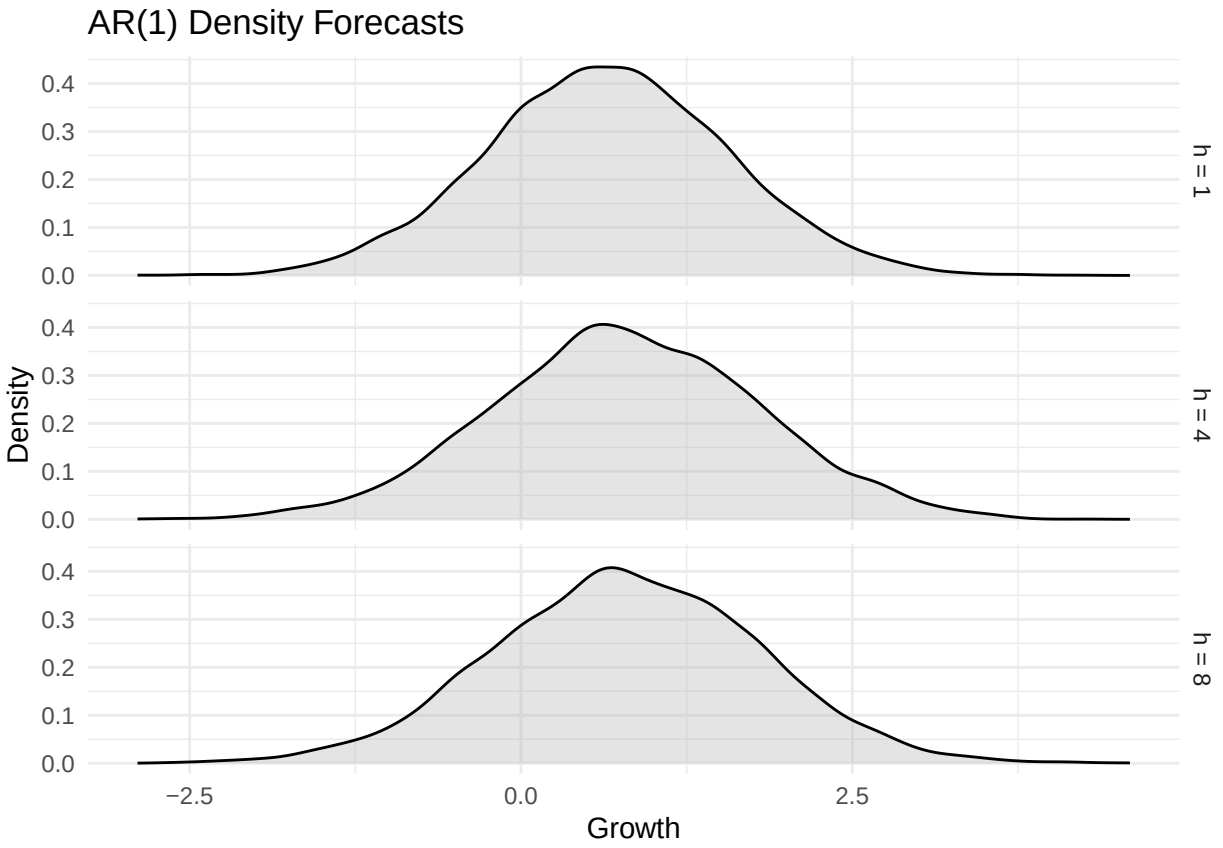
```
##            df      AIC
## fit_ar1     3 705.2926
## fit_ma1     3 714.7341
## fit_arma11  4 706.2247
## fit_arma21  5 705.6231
## fit_arma12  5 704.4493
## fit_arma22  6 702.8533
## fit_ar4     6 704.0614
## fit_ma4     6 706.1358
```

```
BIC(fit_ar1, fit_ma1, fit_arma11, fit_arma21, fit_arma12, fit_arma22, fit_ar4, fit_ma4)
```

```
##            df      BIC
## fit_ar1     3 715.9861
## fit_ma1     3 725.4277
## fit_arma11  4 720.4828
## fit_arma21  5 723.4457
## fit_arma12  5 722.2719
## fit_arma22  6 724.2404
## fit_ar4     6 725.4486
## fit_ma4     6 727.5229
```

```
h <- 8
set.seed(817)
```

```
sims <- replicate(5000, simulate(fit_ar1, nsim = h, future = TRUE))

pick <- c(1,4,8)[c(1,4,8) <= h]   # horizons to show
dens_df <- do.call(rbind, lapply(pick, function(k)
  data.frame(h = paste0("h = ", k), value = sims[k, ])
))

ggplot(dens_df, aes(value)) +
  geom_density(fill = "gray70", alpha = 0.35) +
  facet_grid(rows = vars(h), scales = "free_x") +
  labs(title = "AR(1) Density Forecasts", x = "Growth", y = "Density") +
  theme_minimal()
```



Based on the AIC/BIC figures of a large selection of models, it seems like AR(1) is the best model for this data. Although it isn't the lowest AIC (that would be ARMA(2,2)), all of the AIC numbers are very close together and AR(1) is in the lowest figures ballpark. AR(1) has the lowest BIC, and it is by a relatively wide margin, which seems like more valuable information than AIC.

The density plot, being bell shaped and symmetric, support this conclusion.

8.7

```
data87 <- read_csv("data87.csv")
```

```
## Rows: 5901 Columns: 5
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): OBS
## dbl (4): SP500_OPEN, FTSE_OPEN, R_SP500_OPEN, R_FTSE_OPEN
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
head(data87)
```

```
## # A tibble: 6 x 5
##   OBS    SP500_OPEN FTSE_OPEN R_SP500_OPEN R_FTSE_OPEN
##   <chr>       <dbl>     <dbl>        <dbl>       <dbl>
## 1 1/2/90       353.     2442.       NA          NA
## 2 1/3/90       360.     2451.        1.76        0.364
## 3 1/4/90       359.     2470.       -0.259       0.744
## 4 1/5/90       356.     2442        -0.865      -1.12
## 5 1/8/90       352.     2445.       -0.980       0.127
## 6 1/9/90       354.     2429.        0.462      -0.657
```

```
data87$OBS <- as.Date(data87$OBS, format = "%m/%d/%y")
data87 <- data87[order(data87$OBS), ]

lag1 <- function(x) c(NA, x[-length(x)])

d <- subset(data87, !is.na(R_SP500_OPEN) & !is.na(R_FTSE_OPEN),
            c(OBS, R_SP500_OPEN, R_FTSE_OPEN))
d$SP_l1   <- lag1(d$R_SP500_OPEN)
d$FTSE_l1 <- lag1(d$R_FTSE_OPEN)
d <- d[complete.cases(d), ]

m0 <- lm(R_SP500_OPEN ~ R_FTSE_OPEN, data = d); summary(m0)
```

```
##
## Call:
## lm(formula = R_SP500_OPEN ~ R_FTSE_OPEN, data = d)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -8.5180 -0.4871  0.0291  0.4928  9.1551
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.02150    0.01361    1.58    0.114
## R_FTSE_OPEN  0.50739    0.01152   44.06   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.986 on 5247 degrees of freedom
## Multiple R-squared:   0.27,  Adjusted R-squared:  0.2699
## F-statistic:  1941 on 1 and 5247 DF,  p-value: < 2.2e-16
```

```r
m1 <- lm(R_SP500_OPEN ~ R_FTSE_OPEN + SP_l1, data = d)
m2 <- lm(R_SP500_OPEN ~ R_FTSE_OPEN + SP_l1 + FTSE_l1, data = d)
anova(m1, m2)
```

```
## Analysis of Variance Table
##
## Model 1: R_SP500_OPEN ~ R_FTSE_OPEN + SP_l1
## Model 2: R_SP500_OPEN ~ R_FTSE_OPEN + SP_l1 + FTSE_l1
##   Res.Df    RSS Df Sum of Sq      F Pr(>F)
## 1   5246 5099.6
## 2   5245 5099.6  1  0.005716 0.0059 0.9389
```

Looking at the post-lag regression model, we see that there is strong correlation between FTSE and the SP500. Because British markets open, they should affect American markets, and the data supports this. Although the R^2 value of 0.27 isn't super high, our model insinuates that British markets are responsible for 27% of the variance in American markets 6 hours later.

8.8

```r
data88 <- read_csv("data88.csv")
```

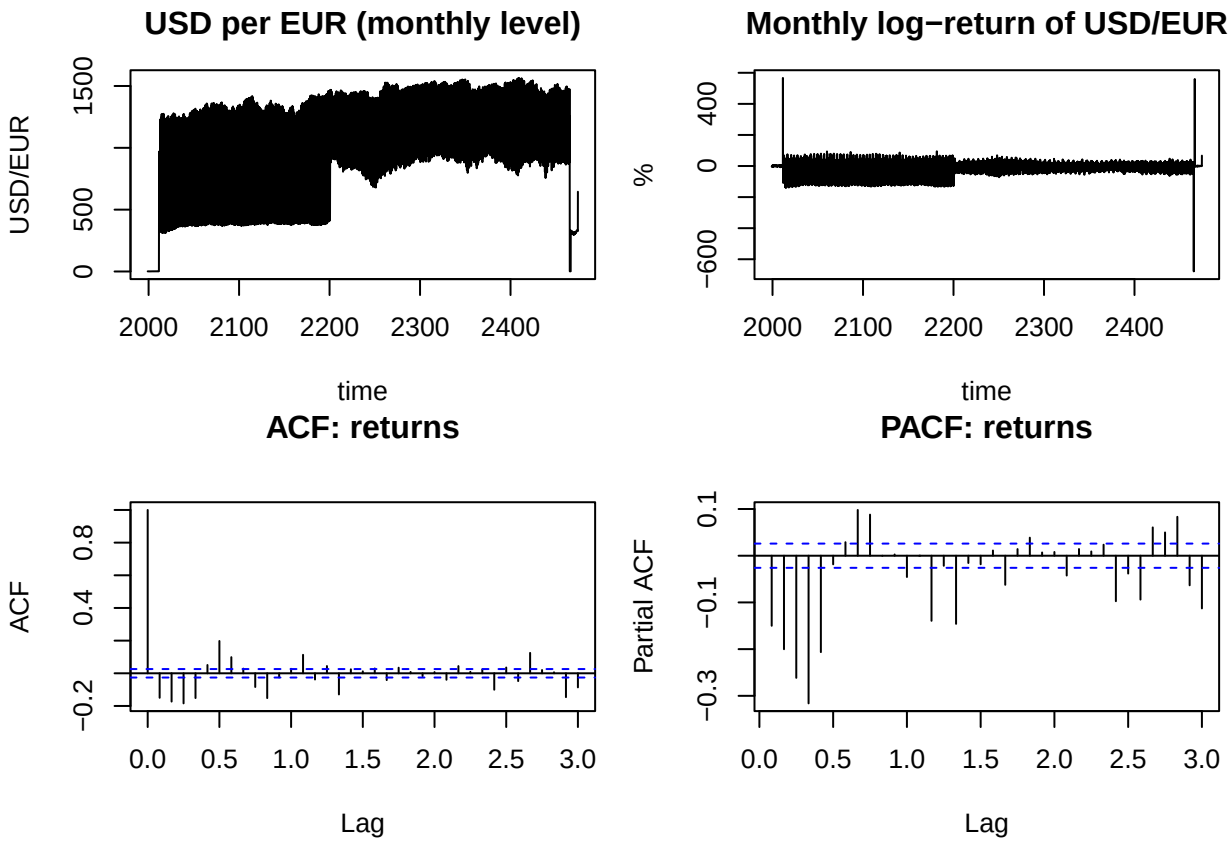
```
## Rows: 5901 Columns: 2
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): DATE
## dbl (1): usd_euro
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
head(data88)
```

```
## # A tibble: 6 x 2
##   DATE   usd_euro
##   <chr>     <dbl>
## 1 1/1/99     1.16
## 2 2/1/99     1.12
## 3 3/1/99     1.09
## 4 4/1/99     1.07
## 5 5/1/99     1.06
## 6 6/1/99     1.04
```

```r
data88 <- subset(data88, !is.na(usd_euro), c(DATE, usd_euro))
data88$DATE <- as.Date(data88$DATE, format = "%m/%d/%y")
data88 <- data88[order(data88$DATE), ]

start_year  <- as.numeric(format(data88$DATE[1], "%Y"))
start_month <- as.numeric(format(data88$DATE[1], "%m"))
x <- ts(data88$usd_euro, start = c(start_year, start_month), frequency = 12)
```

```r
r <- diff(log(x)) * 100
par(mfrow = c(2,2), mar = c(4,4,3,1))
plot.ts(x, main = "USD per EUR (monthly level)", ylab = "USD/EUR", xlab = "time")
plot.ts(r, main = "Monthly log-return of USD/EUR", ylab = "%", xlab = "time")
acf (r, lag.max = 36, main = "ACF: returns")
pacf(r, lag.max = 36, main = "PACF: returns")
```




```r
par(mfrow = c(1,1))

fit_ar1    <- arima(r, order = c(1,0,0), include.mean = TRUE)
fit_ma1    <- arima(r, order = c(0,0,1), include.mean = TRUE)
fit_arma11 <- arima(r, order = c(1,0,1), include.mean = TRUE)

AIC(fit_ar1, fit_ma1, fit_arma11)
```

```
##            df      AIC
## fit_ar1     3 57848.08
## fit_ma1     3 57654.38
## fit_arma11  4 56991.66
```

```r
best_fit <- fit_arma11
fc <- predict(best_fit, n.ahead = 1)
print(fc$pred)
```

```
##             Aug
## 2474 -25.47077
```

We can't predict next month with any accuracy, since this data ends in 2012, but we can forecast 1 month ahead. Using an ARMA(1,1) model due to the low AIC value, we expect the USD to appreciate next month (negative change) relative to the Euro.